

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to $[0,1]$.
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Source Code

Github Repository link: <https://github.com/Kamlesh-DevOP/deep-learning-lab/blob/main/lab-2/Lab%20.ipynb>

8. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

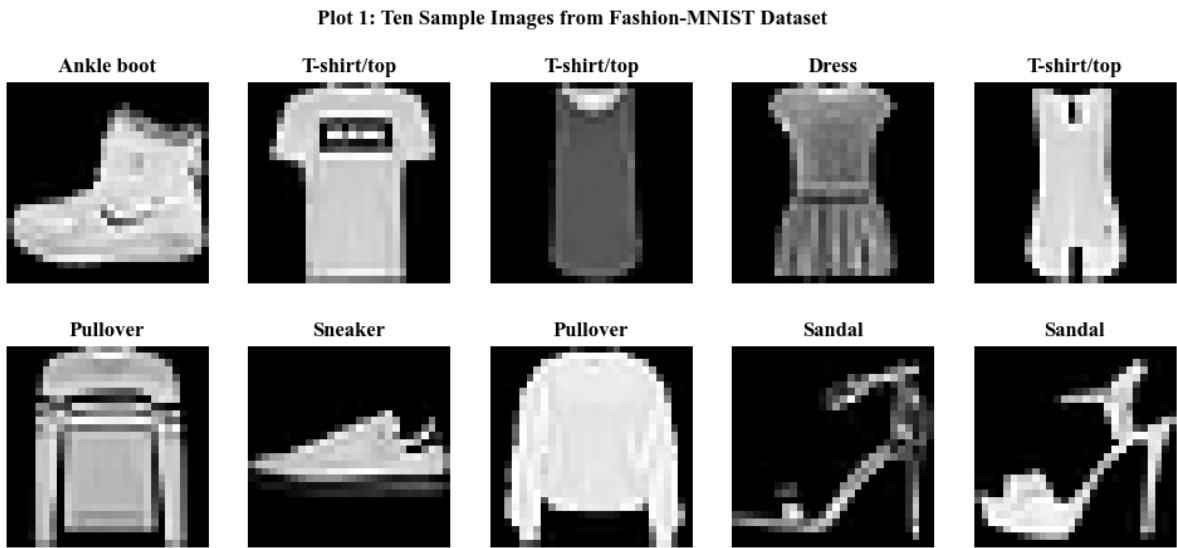
Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

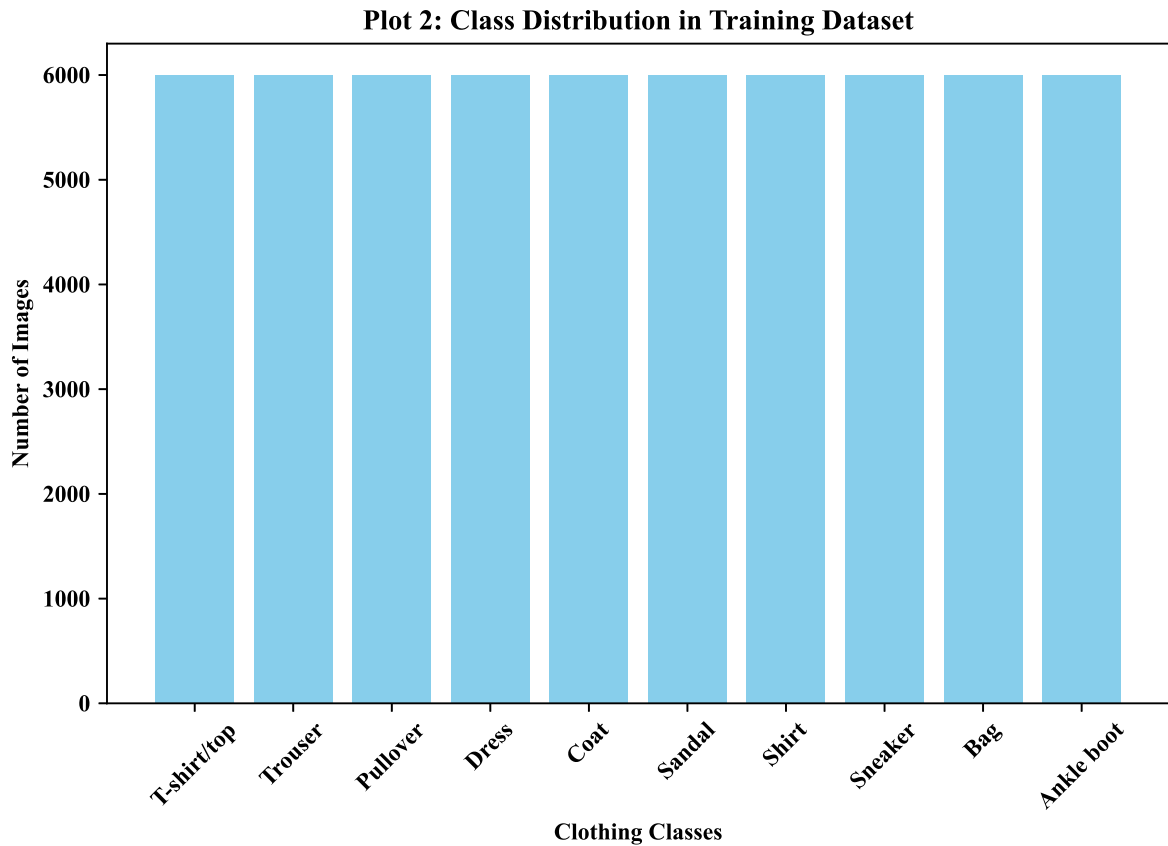
Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

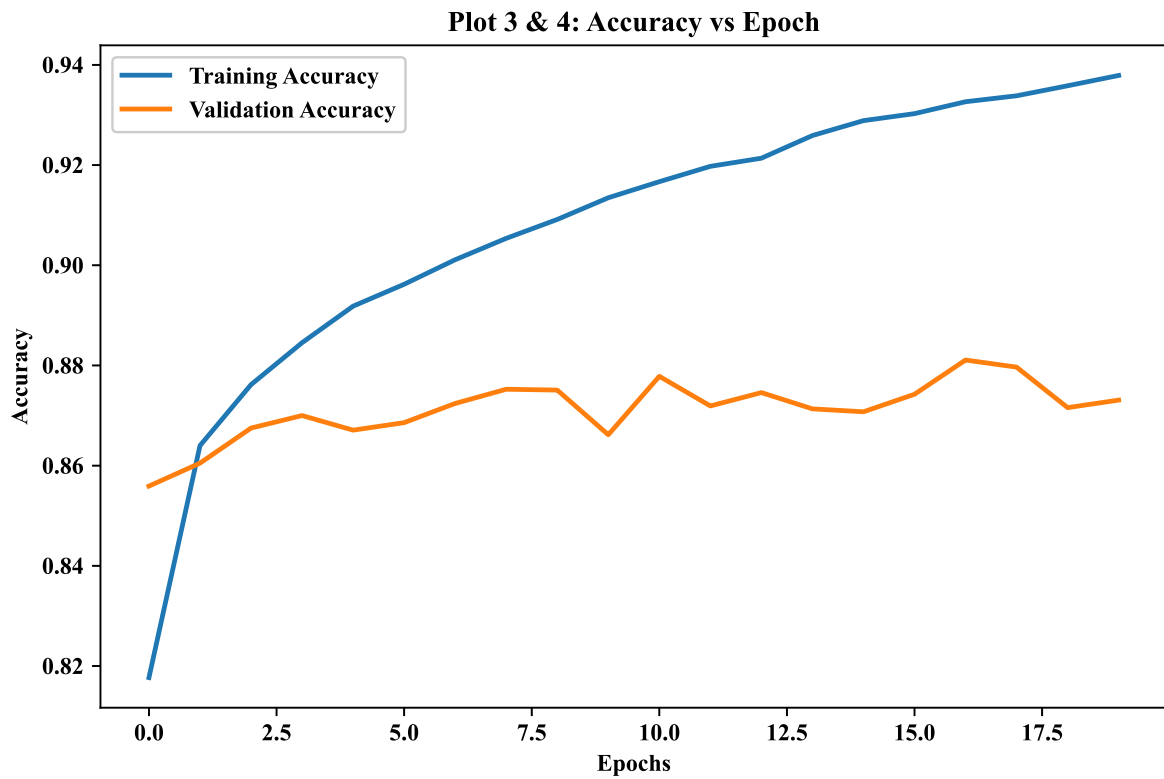
9. Plots with inference



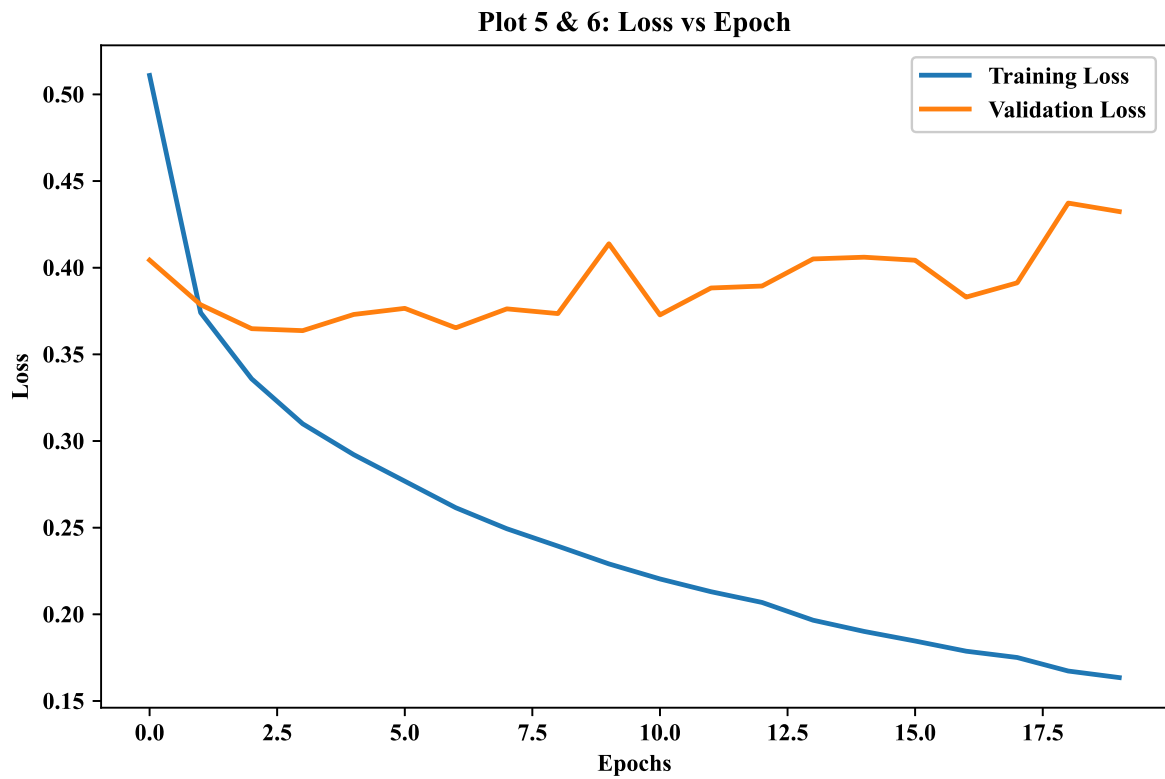
Inference: The dataset consists of low-resolution grayscale images representing ten distinct clothing categories, demonstrating the visual complexity and variations the neural network must learn to interpret.



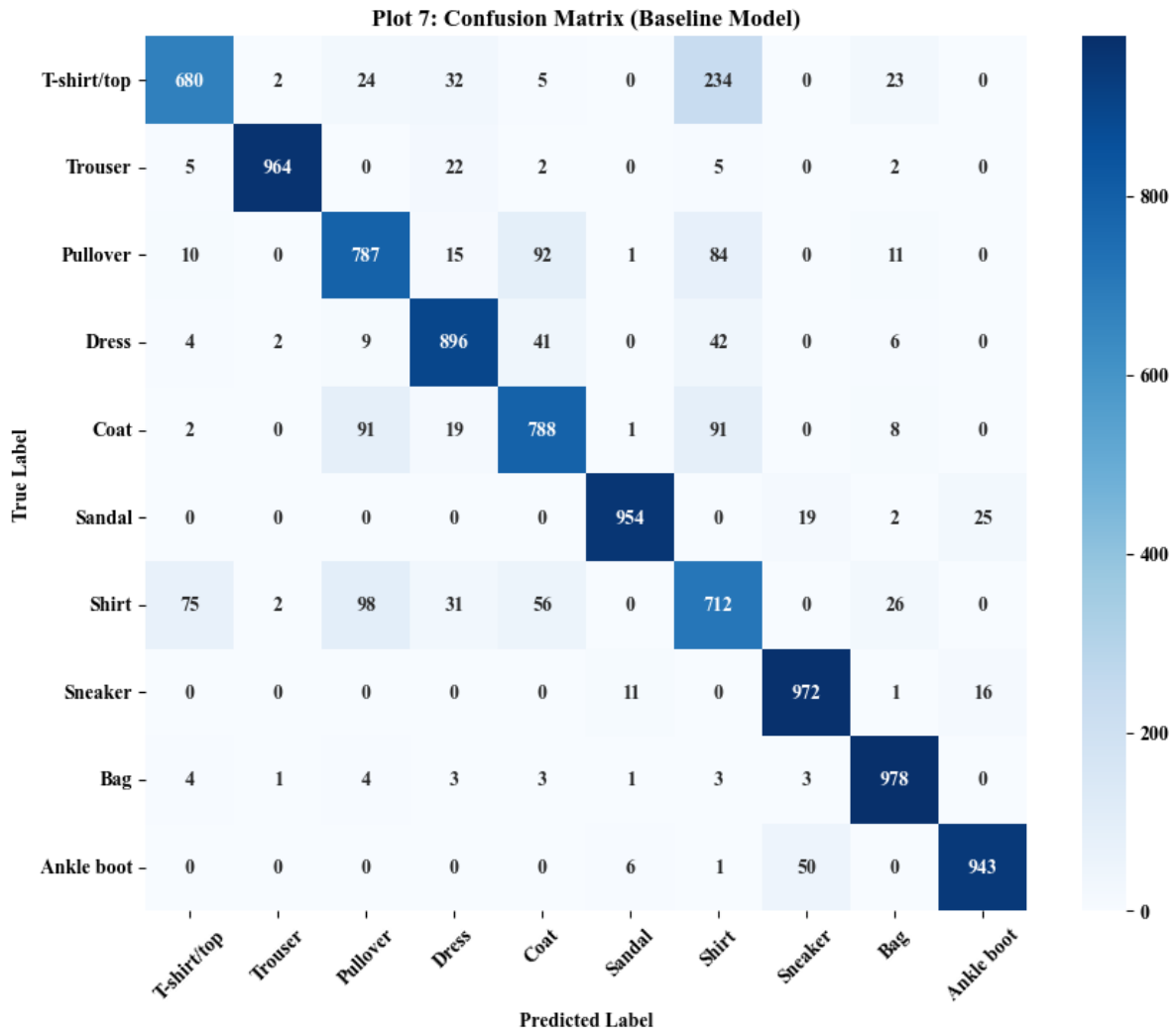
Inference: The training set is perfectly balanced with exactly 6,000 images for each of the 10 classes. This uniform distribution ensures the network will not develop an artificial bias towards any majority class during training.



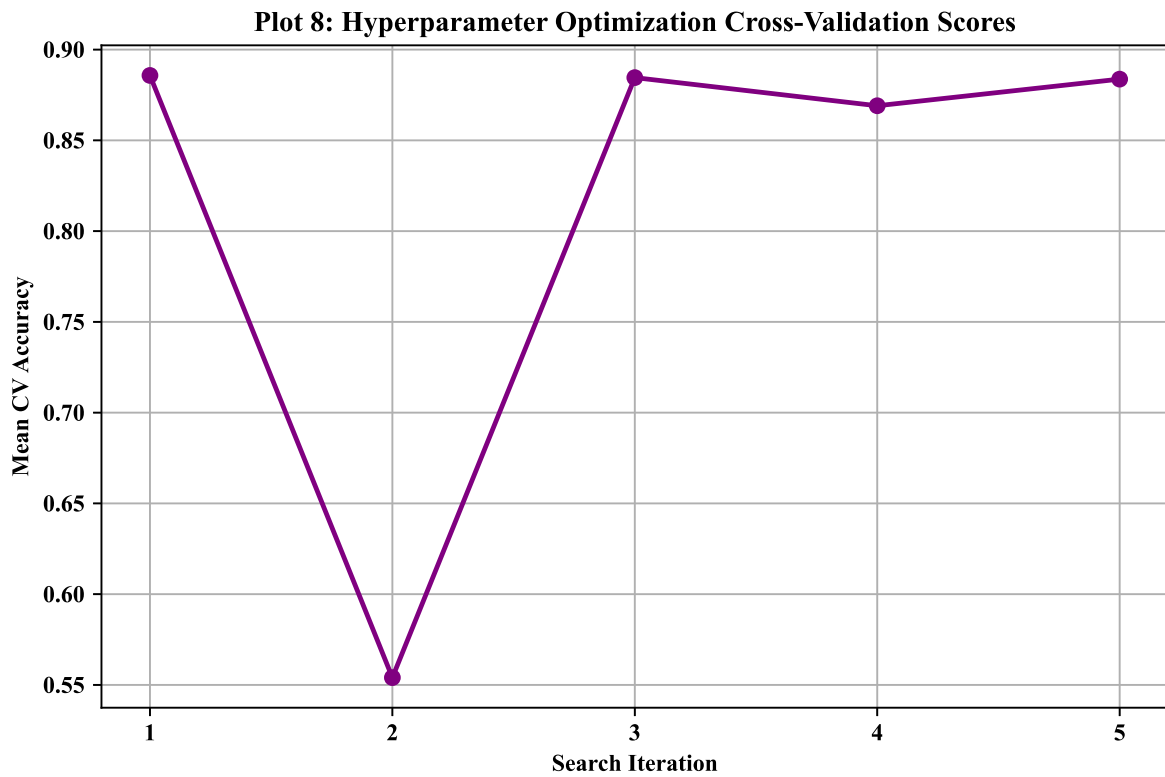
Inference: While the training accuracy consistently climbs toward 94%, the validation accuracy plateaus around 88% after the 10th epoch. This widening performance gap indicates the model is beginning to memorize the training data rather than generalizing to new inputs.



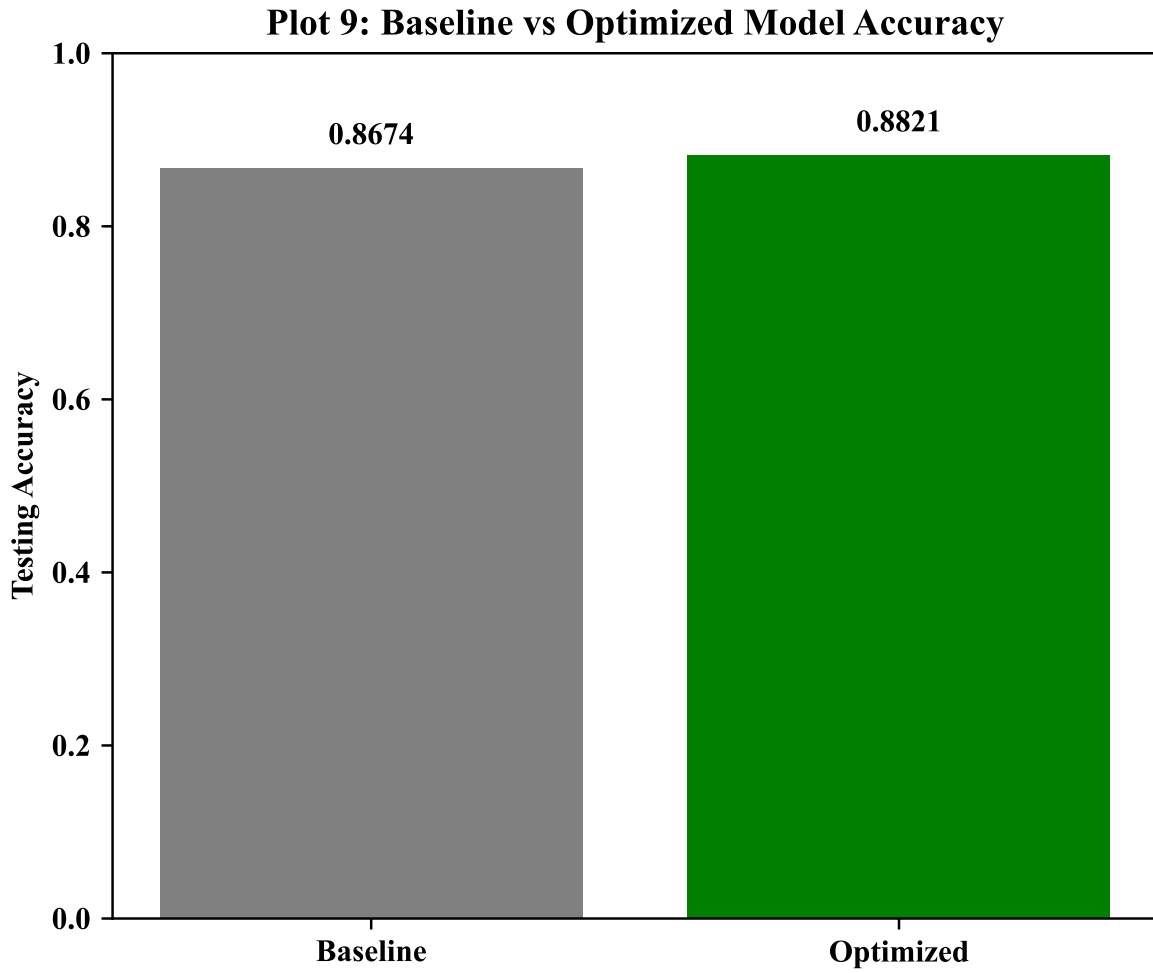
Inference: The training loss steadily approaches zero, but the validation loss bottoms out near epoch 3 and gradually increases thereafter. This distinct divergence confirms the onset of overfitting in the later training stages.



Inference: The strong dark diagonal indicates overall robust classification. However, the model visibly struggles with visually similar upper-body garments, frequently misclassifying shirts as T-shirts or pullovers.



Inference: The sharp plunge in accuracy down to 55% during the second search iteration highlights that arbitrary hyperparameter choices can severely cripple the network, validating the necessity of an automated tuning process.



Inference: The automated tuning successfully identified a superior network configuration, pushing the final testing accuracy from 86.74% in the baseline architecture to an optimized 88.21%.

10. Results

Best Hyperparameters

Hidden Layers	1
Hidden Neurons	256
Learning Rate	0.001
Batch Size	128
Optimizer	Adam
Activation Function	Tanh
Epochs	20
Dropout	0.2
Cross-validation Accuracy	0.8858
Testing Accuracy	0.8821

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8674	0.8821
Precision	0.8722	0.8836
Recall	0.8674	0.8821
F1-score	0.8680	0.8824
Training Time	1.9 minute	1.4 minute

11. Discussion

1. Which optimization method was used?

`RandomizedSearchCV` was utilized alongside 5-fold cross-validation. This method was chosen over exhaustive grid search to efficiently explore the massive hyperparameter space within a reasonable computational timeframe.

2. Which hyperparameters were selected?

The automated search identified the following optimal configuration: a single hidden layer (1) containing 256 neurons, the `Tanh` activation function, the `Adam` optimizer with a learning rate of 0.001, a batch size of 128, a training duration of 20 epochs, and a dropout rate of 0.2.

3. Did optimization improve performance?

Yes, optimization yielded measurable improvements across all metrics. The testing accuracy increased from 86.74% (baseline) to 88.21% (optimized), with corresponding increases in Precision, Recall, and F1-score. Furthermore, the optimization reduced the overall training time from 1.9 minutes to 1.4 minutes.

4. Which hyperparameter had the greatest impact?

The combination of increasing the batch size (from 32 to 128) and reducing the network depth to a single hidden layer had the greatest impact on execution speed, significantly dropping the training time. Meanwhile, substituting the ReLU activations with `Tanh` and introducing a 0.2 Dropout rate had the greatest impact on generalizability, successfully suppressing the overfitting observed in the baseline model.

5. Compare Grid Search and Randomized Search.

Grid Search evaluates every possible hyperparameter combination exhaustively, guaranteeing the discovery of the absolute best configuration within the defined parameters, but it is extremely computationally expensive. Randomized Search evaluates a randomly selected subset of those combinations; while it trades away the guarantee of finding the absolute maximum, it offers a massive speedup and typically discovers a highly optimal model in a fraction of the time.

6. Recommend the best MLP configuration.

Based on the empirical evidence from this experiment, the best configuration for classifying the Fashion-MNIST dataset is a wider, shallower network. The recommended architecture is: an input layer of 784, a single dense layer of 256 neurons with a `Tanh` activation function, followed by a Dropout layer set to 0.2 to prevent memorization, and a 10-neuron Softmax output layer. The network should be compiled with the `Adam` optimizer (learning rate = 0.001) and trained with a batch size of 128.

12. Conclusion

This experiment successfully demonstrated the end-to-end pipeline for designing, training, and optimizing a Multi-Layer Perceptron for image classification on the Fashion-MNIST dataset. While our initial baseline model achieved a respectable accuracy of 86.74%, the validation curves revealed distinct overfitting as the network essentially began memorizing the training data.

By implementing automated hyperparameter tuning through `RandomizedSearchCV`, we were able to systematically resolve this issue. The search revealed that a simpler, shallower architecture—paired with a higher batch size, `Tanh` activations, and a 20% dropout rate—was vastly superior to our deeper baseline.

This optimized configuration not only pushed our final testing accuracy to 88.21%, but it also dropped the total training time down to just 1.4 minutes.

Ultimately, this lab highlighted a core principle of deep learning: blindly adding more layers does not guarantee better performance. Instead, deliberate regularization and intelligent hyperparameter tuning are the true keys to building models that are both highly generalizable and computationally efficient.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.